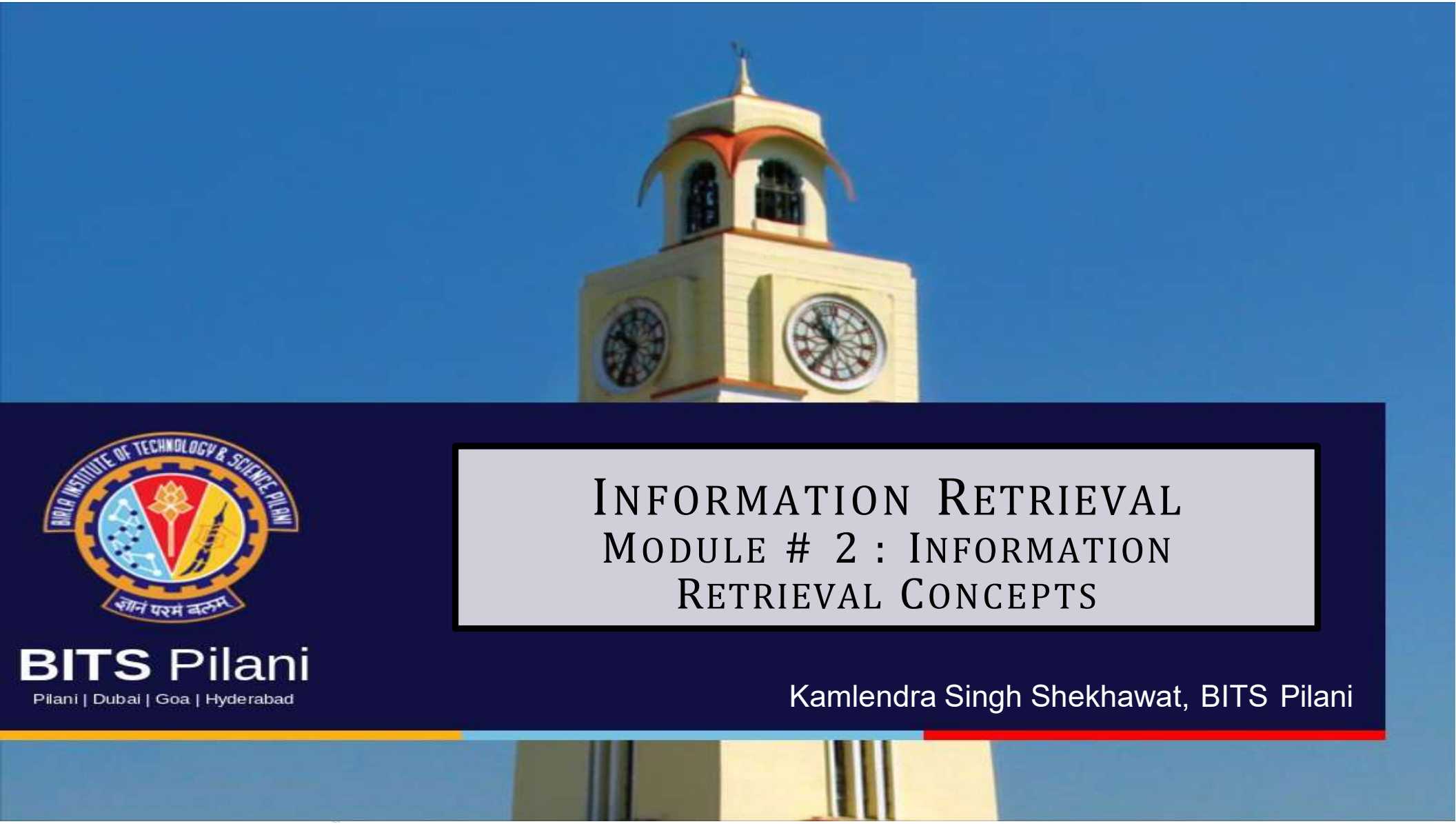


BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Information Retrieval
Module # 2 : Information Retrieval Concepts

Kamlendra Singh Shekhawat, BITS Pilani

The instructor is gratefully acknowledging the authors who made their course materials freely available online.

This deck is prepared by Kamlendra Singh Shekhawat.

Table of Contents

1. Module 2
2. Dictionary and Tolerant Retrieval
3. Dictionary Data Structures
4. Wild-card Queries
5. Spelling Correction

Module Topics....

- Boolean Retrieval
- Dictionary and Tolerant Retrieval
- Index Construction and Compression

Work Integrated Learning Programmes

Table of Contents

1. Module 2
2. Dictionary and Tolerant Retrieval
3. Dictionary Data Structures
4. Wild-card Queries
5. Spelling Correction

INVERTED INDEX

- Given – an inverted index and a query
- Task – determine whether each query term exists in the vocabulary.
- Retrieve – If so, identify the pointer to the corresponding postings.

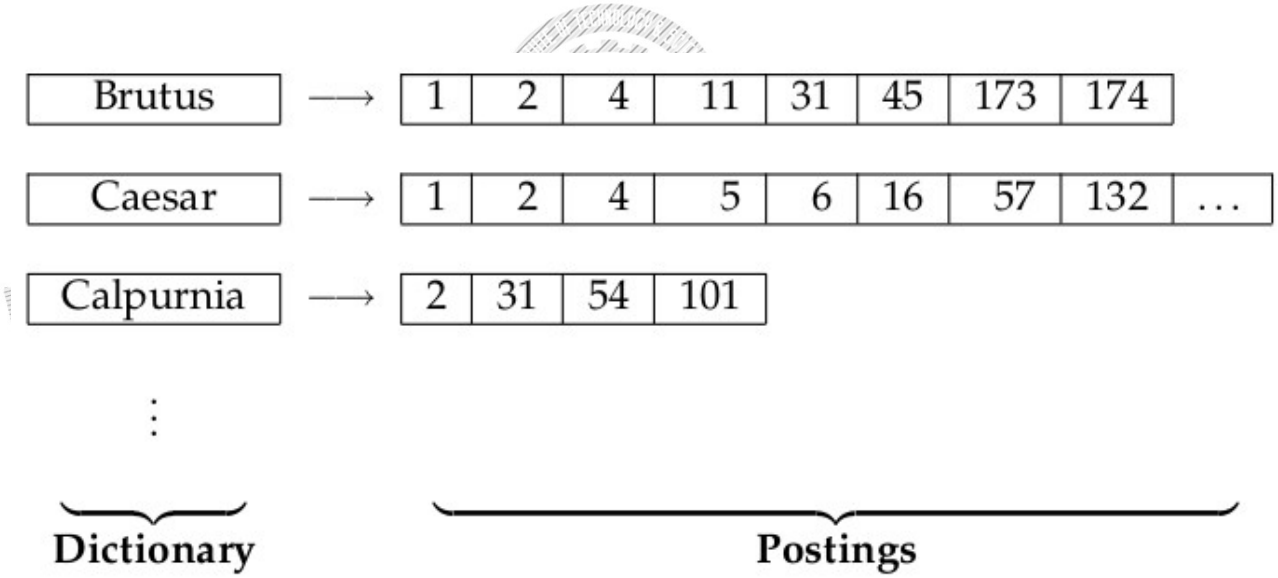


Table of Contents

1. Module 2
2. Dictionary and Tolerant Retrieval
3. Dictionary Data Structures
4. Wild-card Queries
5. Spelling Correction

Dictionary

Dictionary

Vocabulary lookup operation uses a data structure called the dictionary. Dictionary items are called terms or keys.

- How do we store a dictionary in memory efficiently?
- How do we quickly look up elements at query time?
- Which data structure?
  1. How many keys are we likely to have?
  2. Is the number likely to remain static, or change? In the case of changes, will the keys be inserted, deleted or both?
  3. What are the relative frequencies with which various keys will be accessed?

Dictionary Data Structure

- How do we store a dictionary in memory efficiently?
  - Hash tables
  - Search trees

Work Integrated Learning Programmes

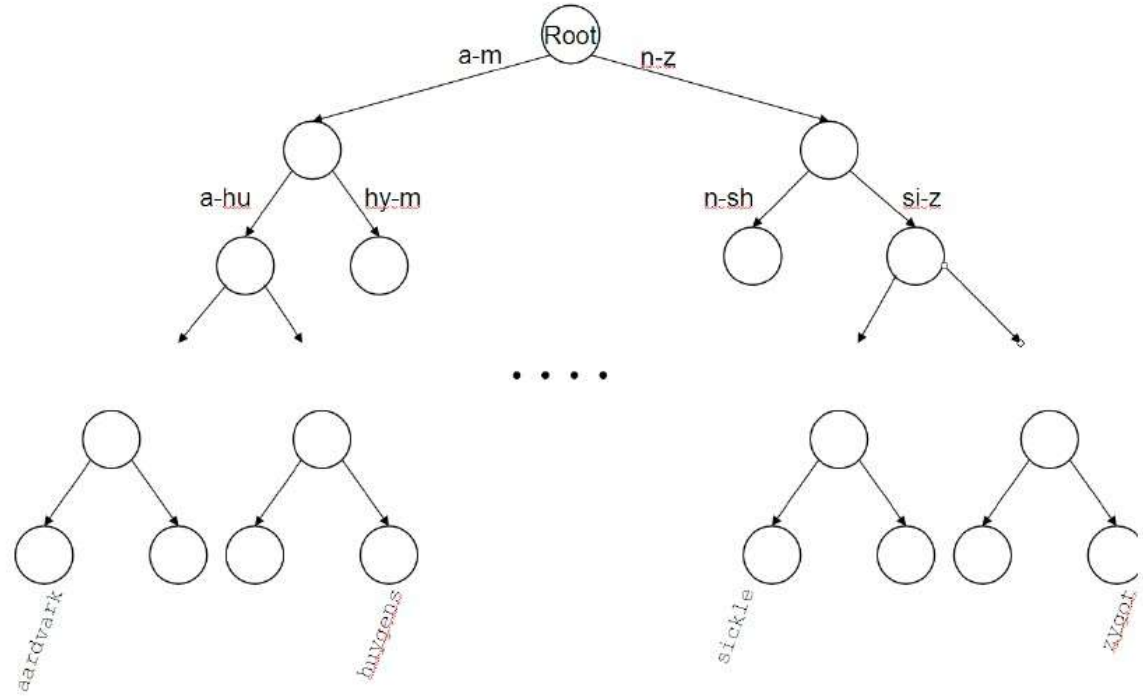
HASHTABLES

- Each vocabulary term is hashed to an integer.
- Query: hash each query term separately and follow a pointer to the corresponding postings.
- Pros
  - Lookup is faster than for a tree: O(1)
- Cons:
  - No easy way to find minor variants: judgment/judgement
  - No prefix search, also called tolerant retrieval.
  - If vocabulary keeps growing, need to occasionally do the expensive operation of rehashing everything.

Tree

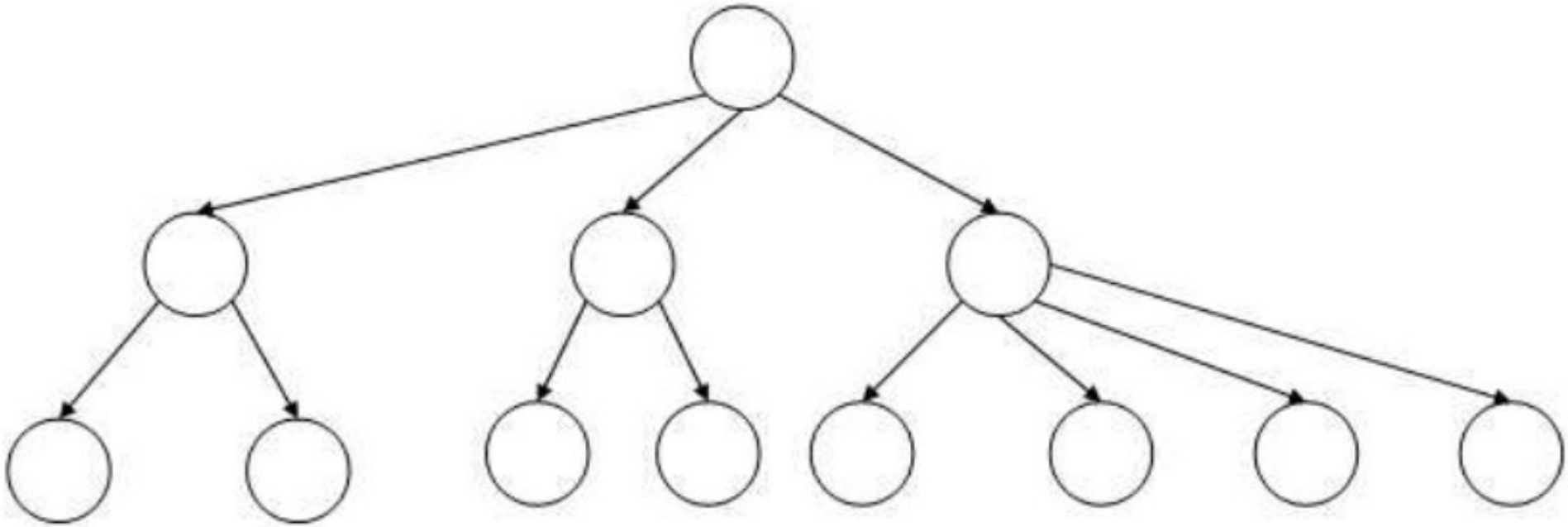
- Simplest: binary tree
- More usual: B-trees
- Trees require a standard ordering of characters and hence strings . . . but we typically have one.
- Pros
  - Solves the prefix problem (terms starting with hyp)
- Cons:
  - Slower: O(log M) [and this requires balanced tree]
  - Rebalancing binary trees is expensive.
  - B-trees mitigate the rebalancing problem.

Tree: Binary Tree



The branch at the root partitions vocabulary terms into two subtrees, those whose first letter is between a and m, and the rest.

Tree: B-Tree



In B-tree, every internal node has between 2 and 4 children. Every internal node has a number of children in the interval [a,b] where a, b are appropriate natural numbers, e.g., [2,4].

Table of Contents

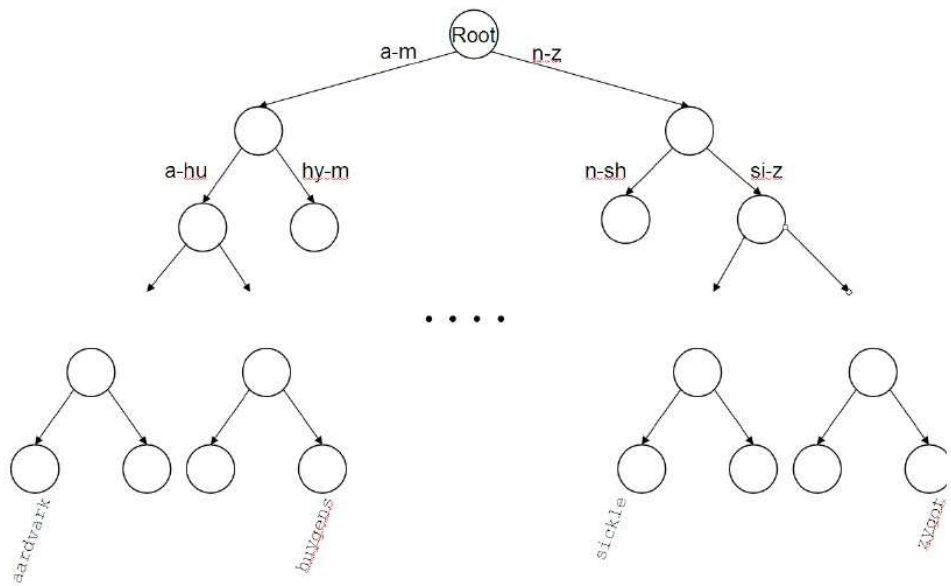
1. Module 2
2. Dictionary and Tolerant Retrieval
3. Dictionary Data Structures
4. Wild-card Queries
5. Spelling Correction

Wild-card Queries

- The user is uncertain of the spelling of a query term. Sydney vs. Sidney
- The user is aware of multiple variants of spelling a term and (consciously) seeks documents containing any of the variants. color vs. colour
- The user seeks documents containing variants of a term that would be caught by stemming, but is unsure whether the search engine performs stemming. judicial vs. judiciary -- judicia*
- The user is uncertain of the correct rendition of a foreign word or phrase. Universit* Stuttgart

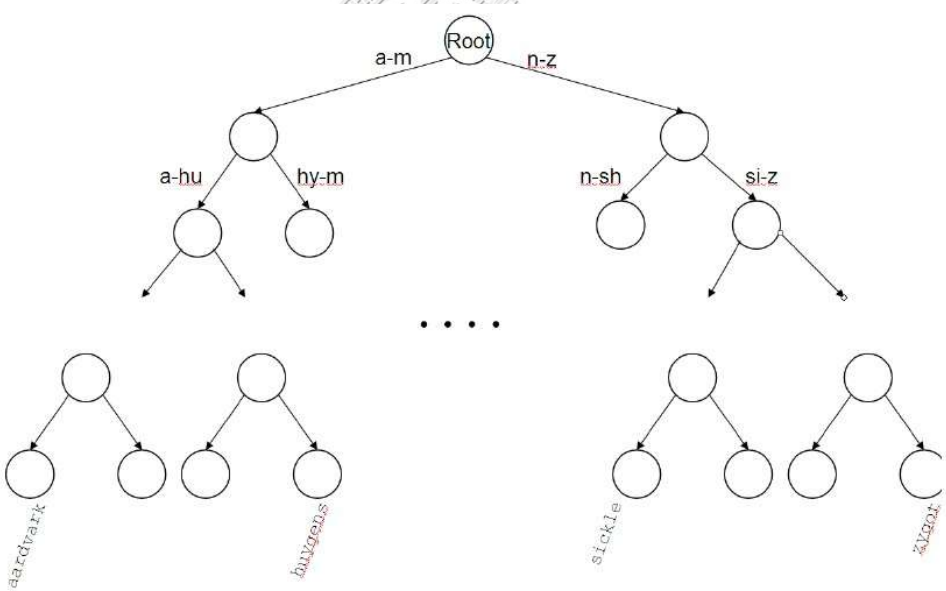
Wild-card queries: * suffix

- mon*: find all docs containing any word beginning with mon.
- Easy with binary tree (or B-tree) lexicon.
- Retrieve all words in range: $\text{mon} \leq W < \text{moo}$
- $|W|$ lookups on the standard inverted index.



Wild-card queries: * prefix

- *mon: find words ending in mon
- Harder
- Maintain an additional B-tree for terms backwards.
- Retrieve all words in range: $nom \leq w < non$



Query Processing

- An enumeration of all terms in the dictionary that match the wild-card query.
- Look up the postings for each enumerated term.
- Consider the query:
  `se*ate AND fil*er`
  This may result in the execution of many Boolean AND queries.

EXERCISE

How can we enumerate all terms meeting the wild-card query `fil*er` ?
Answer

- Use the regular B-tree to enumerate the set W of dictionary terms beginning with the prefix `fil`.
- Reverse B-tree to enumerate the set R of terms ending with the suffix `er`.
- Intersection $W \cap R$ of these two sets to arrive at the set of terms that begin with the prefix `fil` and end with the suffix `er`.
- Use the standard inverted index to retrieve all documents containing any terms in this intersection.

MIDDLE *

- How can we handle * in the middle of query term?
- To handle wildcard queries that contain a single * symbol, use two B-trees, the normal B-tree and a reverse B-tree. Use the standard inverted index to retrieve all documents containing any terms in this intersection.
- Expensive
- Alternate: transform wild-card queries so that the * occur at the end. This gives rise to the Permuterm Index.

Permuterm Index

- Special symbol $ to mark the end of a term.
  `hello -- hello$`
- Construct a permuterm index, in which the various rotations of each term (augmented with $) all link to the original vocabulary term.
- Set of rotated terms in the permuterm index is known as the permuterm vocabulary.
- For term `hello`, index under:
  `hello$, ello$h, llo$he, lo$hel, o$hell.`

Permuterm Index Queries

- Rotate query wild-card to the right. Use B-tree lookup as before.
- Con: quadruples lexicon size for English.
- Queries
  - X lookup on X$
  - X* lookup on $X*
  - *X lookup on X$*
  - *X* lookup on X*
  - X*Y lookup on Y$X*
- Query = hel*o

  X = hel, Y=o

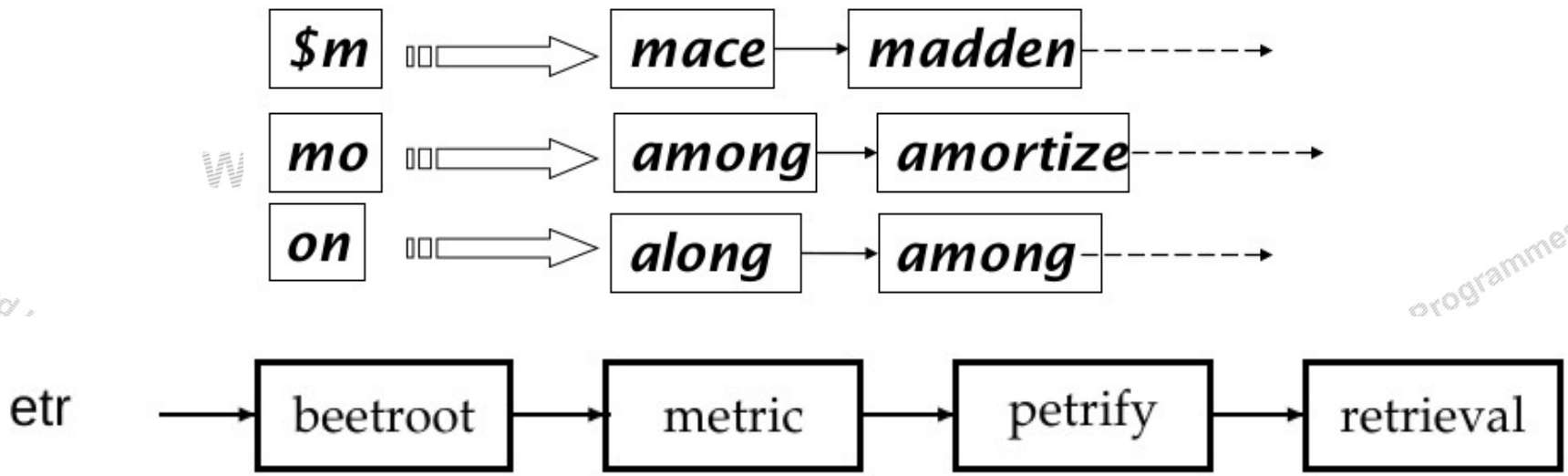
  Lookup o$hel*

Bigram (k-gram) indexes

- Enumerate all k-grams (sequence of k chars) occurring in any term.
- A k-gram is a sequence of k characters.
- Use a special character $ to denote the beginning or end of a term.
  3-grams generated for `castle` – `$ca, cas, ast , stl, tle, le$`

Bigram (k-gram) indexes

- In a k-gram index, the dictionary contains all k-grams that occur in any term in the vocabulary. i.e. Maintain a second inverted index from k-grams to dictionary terms that match each k-gram.
- Each postings list points from a k-gram to all vocabulary terms containing that k-gram.



BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

K-GRAMS AND WILD CARD QUERIES

- Gets terms that match and intersect.
- Query mon* can now be run as $m AND mo AND on results in moon also.
- Requires post-filter these terms against query.
- Surviving enumerated terms are then looked up in the term-document inverted index.
- Adv: Fast, space efficient (compared to permuterm).

Wild card in Search Engines

- Search engines do allow the combination of wildcard queries using Boolean operators, for example, `re*d AND fe*ri`.
- Conjunction of disjunctions: we seek all documents that contain any term matching `re*d` and any term matching fe*ri.
- The processing of a wildcard query can be quite expensive, because of the added lookup in the special index, filtering and finally the standard inverted index.

Table of Contents

1. Module 2
2. Dictionary and Tolerant Retrieval
3. Dictionary Data Structures
4. Wild-card Queries
5. Spelling Correction

Correcting Spelling Errors

- Correcting document(s) being indexed.
- Retrieve documents containing the term `carrot` when the user types the query `carot`.
- Two steps to solving this problem:
  - based on edit distance
  - based on k-gram overlap

Correcting Spelling Errors

- Correcting document(s) being indexed.
- Retrieve documents containing the query term.
- Two main flavours :
  - Isolated word
    - Check each word on its own for misspelling.
    - Will not catch typos resulting in correctly spelled words
      `form → from carot → carrot.`
  - Context-sensitive
    - Look at surrounding words,
      `I flew form Heathrow to Narita.`

Document Correction

- Especially needed for OCR'ed documents
  - Correction algorithms are tuned for this.
  - use domain-specific knowledge
  - OCR can confuse O and D more often than it would confuse O and I (adjacent on the keyboard, so more likely interchanged in typing).
- Web pages and printed material have typos.
- Goal: the dictionary contains fewer misspellings.
- Often we don't change the documents and instead fix the query-document mapping.

Query Misspellings

Query a r rahaman

- Option 1: Retrieve documents indexed by the correct spelling.
- Option 2: Return several suggested alternative queries with the correct spelling.

  Did you mean . . . ?

Isolated Word Correction

- Fundamental premise – there is a lexicon from which the correct spellings can be arrived at.
- Two basic choices for this
  - A standard lexicon such as
    - Webster's English Dictionary
    - An 'industry-specific' lexicon - hand-maintained
  - Lexicon of the indexed corpus
    - All words on the web
    - All names, acronyms etc. (Including the misspellings)

Isolated Word Correction

- Given a lexicon and a character sequence Q, return the words in the lexicon closest to Q.
- What's "closest"?
  - Edit distance (Levenshtein distance)
  - Weighted edit distance
  - n-gram overlap

Edit Distance

- Given two character strings s_1 and s_2 , the edit distance between them is the minimum number of edit operations required to transform s_1 into s_2.
- Edit operations allowed for this purpose
  - insert a character into a string
  - delete a character from a string
  - replace a character of a string by another character
- Edit distance is sometimes known as Levenshtein distance.
- Compute is $O(|s_1| \times |s_2|)$, where $|s_i|$ denotes the length of a string s_i .
- Generally found by dynamic programming.
- Edit distance

  `dof to dog` is 1

  `cat to act` is 2 (Just 1 with transpose.)

  `cat to dog is 3`

EXERCISE

Find the edit distance.

- Input: str1 = ”geek”, str2 = ”gesek”
  Output: 1 (insert a ’s’)
- Input: str1 = ”cat”, str2 = ”cut”
  Output: 1 (Replace ’a’ with ’u’)
- Input: str1 = ”sunday”, str2 = ”saturday”
  Output: 3 (Replace ’n’ with ’r’, insert t, insert a)

Weighted Edit Distance

- Allow different weights for different kinds of edit operations
- Weight of an operation depends on the character(s) involved.
  - Meant to capture OCR or keyboard errors.
  - m more likely to be mistyped as n than as q.
  - replacing m by n is a smaller edit distance than by q.
  - This may be formulated as a probability model.
- Requires weight matrix as input.
- Modify dynamic programming to handle weights.

Using Edit Distances

- Given query, first enumerate all character sequences within a preset (weighted) edit distance (e.g., 2)
- Intersect this set with list of ”correct” words.
- Show terms you found to user as suggestions.
  - We can look up all possible corrections in our inverted index and return all docs – slow
  - We can run with a single most likely correction.
- The alternatives disempower the user, but save a round of interaction with the user.

Edit Distance to all Dictionary Terms?

- Given a (misspelled) query, do we compute its edit distance to every dictionary term?
  - Expensive and slow
- Alternative?
  - How do we cut the set of candidate dictionary terms?
  - One possibility is to use n-gram overlap for this.
  - This can also be used by itself for spelling correction.

N-gram Overlap

- Enumerate all the n-grams in the query string as well as in the lexicon.
- Use the n-gram index (recall wild-card search) to retrieve all lexicon terms matching any of the query n-grams.
- Threshold by number of matching n-grams. Variants - weight by keyboard layout, etc.

Example with Trigrams

- Text is november
- Trigrams are nov, ove, vem, emb, mbe, ber
- The query is december
- Trigrams are dec, ece, cem, emb, mbe, ber
- So 3 trigrams overlap (of 6 in each term).
- How can we turn this into a normalized measure of overlap?

Jaccard Coefficient

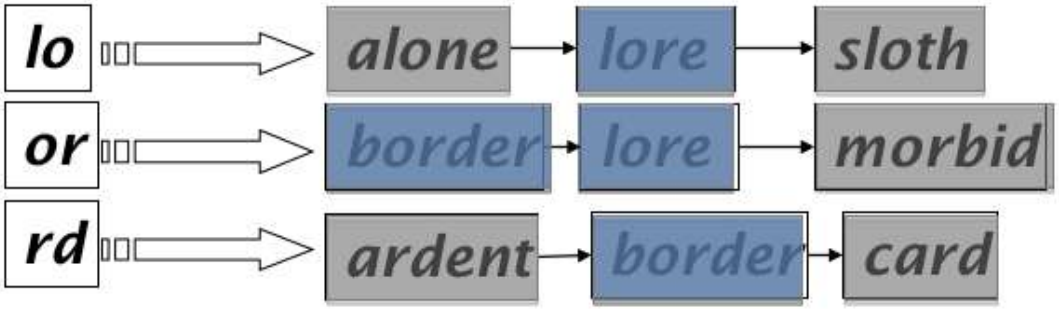
- commonly-used measure of overlap
- Let X and Y be two sets

$$\text{Jaccard Coefficient} = \frac{|X \cap Y|}{|X \cup Y|}$$

- Jaccard Coefficient = 1 when X and Y have the same elements.
- Jaccard Coefficient = 0 when X and Y are disjoint.
- X and Y don't have to be of the same size.
- Always assigns a number between 0 and 1.
- Threshold to decide if you have a match E.g., if $J.C. > 0.8$, declare a match.

Matching Bigrams

- Query `lord` we wish to identify words matching 2 of its 3 bigrams `lo`, `or`, `rd`



Standard postings "merge" will enumerate …

Adapt this to using Jaccard (or another) measure.

Context-sensitive Spell Correction

- Text: I flew from Heathrow to Narita.
- Consider the phrase query flew form Heathrow
- We'd like to respond
  Did you mean "flew from Heathrow"?
  because no docs matched the query phrase.

Context-sensitive Spell Correction

First idea

- Retrieve dictionary terms close (in weighted edit distance) to each query term.
- Need surrounding context to catch this.
- Now try all possible resulting phrases with one word ”fixed” at a time.
  flew from heathrow
  fled form heathrow
  flea form heathrow
- Hit-based spelling correction: Suggest the alternative that has lots of hits.

Context-sensitive Spell Correction

Second Idea

- Break phrase query into a conjunction of biwords.
- Look for biwords that need only one term corrected.
- Enumerate only phrases containing ”common” biwords.

References

1. Chapter 3
   C. D. Manning, P. Raghavan and H. Schutze. Introduction to Information Retrieval. 2008.
   http://nlp.stanford.edu/IR-book/

Thank You!